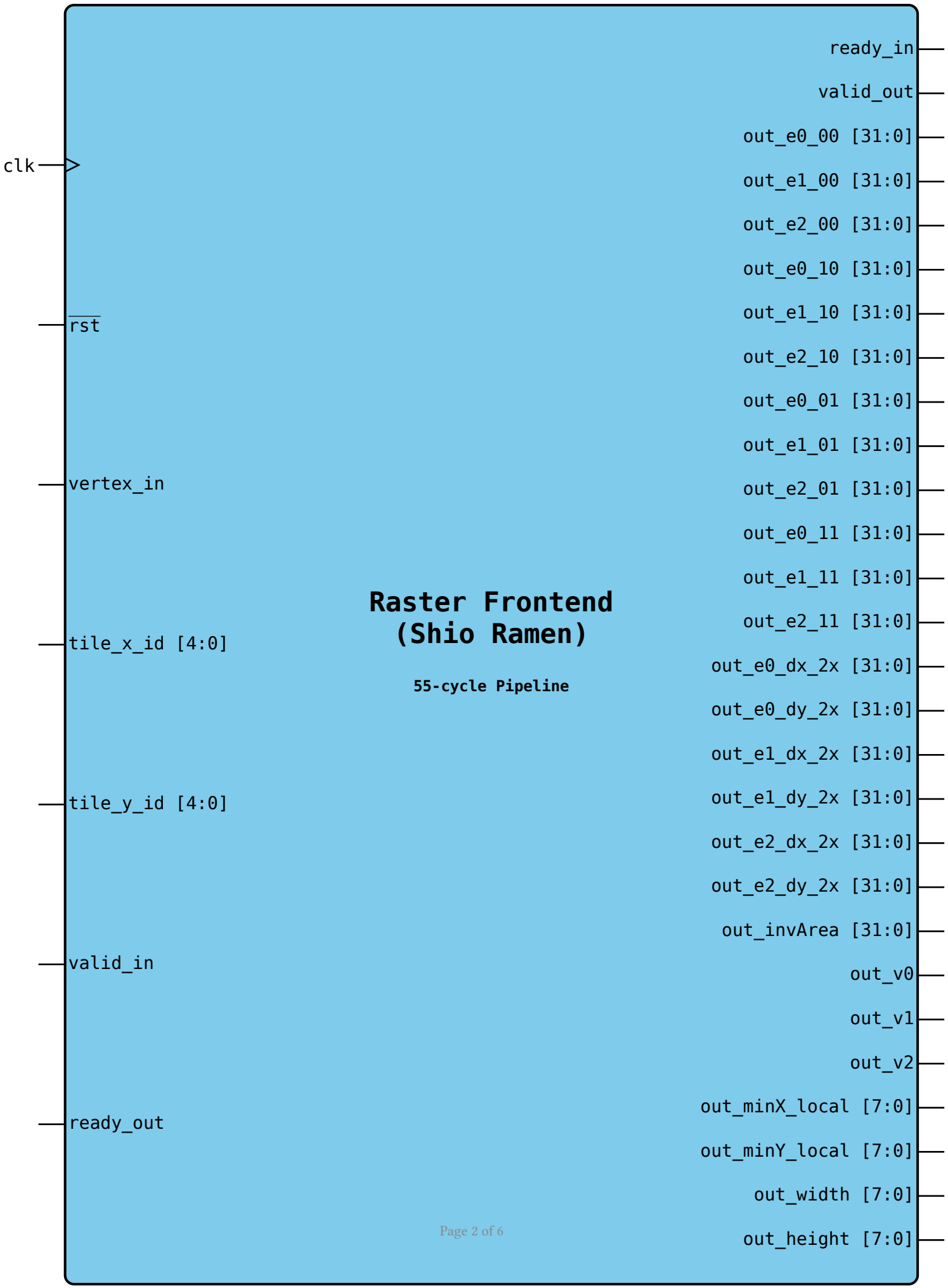
Raster Core Architecture

Shio Ramen Edge Function & Inverse Area Pipeline

1. Module Overview

The `raster_core` module is responsible for computing the edge functions and the inverse triangle area for a given triangle and tile. It utilizes a highly optimized sequential pipeline that shares floating-point subtraction, multiplication, and reciprocal arithmetic units to calculate the values over a period of 62 cycles. To optimize initial coordinate calculations, the design employs an internal Lookup Table (LUT) to convert the integer tile IDs (`tile_x_id`, `tile_y_id`) into floating-point coordinates quickly, avoiding the hardware overhead and latency of a dynamic floating-point conversion unit.

The block diagram below details the module's input and output ports:


Raster Frontend
(Shio Ramen)
55-cycle Pipeline
clk
rst
vertex_in
tile_x_id [4:0]
tile_y_id [4:0]
valid_in
ready_out
ready_in
valid_out
out_e0_00 [31:0]
out_e1_00 [31:0]
out_e2_00 [31:0]
out_e0_10 [31:0]
out_e1_10 [31:0]
out_e2_10 [31:0]
out_e0_01 [31:0]
out_e1_01 [31:0]
out_e2_01 [31:0]
out_e0_11 [31:0]
out_e1_11 [31:0]
out_e2_11 [31:0]
out_e0_dx_2x [31:0]
out_e0_dy_2x [31:0]
out_e1_dx_2x [31:0]
out_e1_dy_2x [31:0]
out_e2_dx_2x [31:0]
out_e2_dy_2x [31:0]
out_invArea [31:0]
out_v0
out_v1
out_v2
out_minX_local [7:0]
out_minY_local [7:0]
out_width [7:0]
out_height [7:0]

2. Module Interface

The table below specifies the functionality and description of each port on `raster_core`:

Port Name	Direction	Type / Width	Description
clk	Input	logic	System Clock (100 MHz)
rst_n	Input	logic	System Reset (Active Low)
vertex_in	Input	vertex_t	Sequential Vertex coordinate input (X/Y)
tile_x_id	Input	logic [8:0]	X coordinate ID of the tile
tile_y_id	Input	logic [8:0]	Y coordinate ID of the tile
valid_in	Input	logic	Input Handshake Valid signal
ready_in	Input	logic	Downstream Handshake Ready signal
ready_out	Output	logic	Input Handshake Ready signal (asserted in `ST_READ_*`)
valid_out	Output	logic	Output Handshake Valid signal (asserted in ST_STALL)
out_e0_row	Output	logic [31:0]	Evaluated edge function 0 at tile corner
out_e1_row	Output	logic [31:0]	Evaluated edge function 1 at tile corner
out_e2_row	Output	logic [31:0]	Evaluated edge function 2 at tile corner
out_e0_dx, out_e0_dy	Output	logic [31:0]	Edge function 0 X and Y derivatives
out_e1_dx, out_e1_dy	Output	logic [31:0]	Edge function 1 X and Y derivatives
out_e2_dx, out_e2_dy	Output	logic [31:0]	Edge function 2 X and Y derivatives
out_invArea	Output	logic [31:0]	Inverse triangle area (floating-point recip)
out_v0, out_v1, out_v2	Output	vertex_t	Registered output copy of vertex coordinates

3. Mathematical Model

For a triangle defined by vertices v_0, v_1, v_2, and a tile corner coordinate $p_{\min}$, the rasterizer evaluates the edge functions and inverse area using a unified edge function helper.

1. **Edge Function Definition:** The core mathematical primitive is the 2D edge function $E(a, b, c)$, which determines the signed area of the triangle formed by a, b, and c, or the position of point c relative to the directed edge $a \to b$:

$$E(a, b, c) = (c.x - a.x) \times (b.y - a.y) - (c.y - a.y) \times (b.x - a.x)$$

2. **Tile Corner Coordinate ($p_{\min}$):** The tile start coordinate $p_{\min} = (p_{\min}.x, p_{\min}.y)$ represents the center of the top-left pixel in the tile. It is computed from the integer tile IDs using a $+0.5$ half-pixel offset to align with the pixel center grid:

$$p_{\min}.x = (\text{tile_x_id} \times \text{TILE_SIZE}) + 0.5$$

$$p_{\min}.y = (\text{tile_y_id} \times \text{TILE_SIZE}) + 0.5$$

3. **Triangle Area:**

$$\text{area} = E(v_0, v_1, v_2)$$

4. **Inverse Area:**

$$\text{invArea} = \frac{1.0}{\text{area}}$$

5. **Edge Evaluation at Tile Corner (p_{min}):**

$$e_0 = E(v_1, v_2, p_{\text{min}})$$

$$e_1 = E(v_2, v_0, p_{\text{min}})$$

$$e_2 = E(v_0, v_1, p_{\text{min}})$$

6. **Edge Derivatives & Step-Based Increments:** The derivatives $e_{i,\text{ dx}}$ and $e_{i,\text{ dy}}$ represent the partial derivatives of the edge function $E(v_i, v_{i+1}, p)$ with respect to $p.x$ and $p.y$:

$$e_{i,\text{ dx}} = \frac{\partial E(v_i, v_{i+1}, p)}{\partial p.x} = v_{i+1}.y - v_i.y$$

$$e_{i,\text{ dy}} = \frac{\partial E(v_i, v_{i+1}, p)}{\partial p.y} = -\big(v_{i+1}.x - v_i.x\big) = v_i.x - v_{i+1}.x$$

These derivatives describe the rate of change of the edge function when moving by a single pixel in the x and y directions. This enables the downstream pixel shader/shader block to evaluate the edge equations at any grid coordinate $(x+1, y)$ or $(x, y+1)$ incrementally using low-cost additions instead of re-evaluating the full edge equation:

$$e_i(x+1, y) = e_i(x, y) + e_{i,\text{ dx}}$$

$$e_i(x, y+1) = e_i(x, y) + e_{i,\text{ dy}}$$

6. Control Flow & Stalling Behavior

The raster core FSM operates as follows:

- **ST_READ_V0 / ST_READ_V1 / ST_READ_V2**: The FSM waits for `valid_in = 1` to read the three sequential vertices of the triangle (`v0`, `v1`, `v2`). `ready_out` is high.
- **ST_CALC**: Once all three vertices are received, the FSM transitions to `ST_CALC` and `cycle_cnt` increments up to 62. `ready_out` is low.
- **ST_STALL**: At `cycle_cnt == 62`, the pipeline results are latched, `valid_out` goes high to indicate the outputs are valid, and the FSM unconditionally transitions to `ST_STALL`.
  - While in `ST_STALL`, `ready_out` remains `0` (so the core blocks and refuses new vertices).
  - The core remains in `ST_STALL` holding the valid output data until `ready_in` goes high, indicating that the downstream consumer has read the outputs.
  - When `ready_in` is asserted, `valid_out` goes back to `0`, and the FSM transitions to `ST_READ_V0` to prepare for the next triangle.